

Informatyka Stosowana

Zaawansowane Technologie Internetowe

Aplikacja internetowa do współdzielonego planowania posiłków i organizacji zakupów spożywczych

Dokumentacja funkcjonalna i wdrożeniowa

Autor: Jakub Gałka
Projekt: WSP
Technologie: Java Spring Boot, Flutter, PostgreSQL, Docker

16 czerwca 2026

Spis treści

1	Cel i zakres aplikacji	2
2	Opis funkcjonalności aplikacji	2
2.1	Zarządzanie użytkownikami	2
2.2	Grupy użytkowników	2
2.3	Planowanie posiłków	3
2.4	Lista zakupów	3
2.5	Przepisy	3
2.6	Rejestrowanie zdarzeń systemowych przez Spring AOP	3
3	Część serwerowa	3
3.1	Spring Boot REST API	3
3.2	Spring Data JPA i Hibernate	4
3.3	PostgreSQL	4
3.4	JWT i Spring Security	4
3.5	Spring AOP do logowania zdarzeń	4
4	Część kliencka	5
4.1	Flutter	5
4.2	Komunikacja z REST API	5
4.3	Logowanie i przechowywanie tokena JWT	5
4.4	Podstawowe ekrany aplikacji	5
5	Prezentacja wybranych testów	6
5.1	Testy backendu	6
5.2	Testy klienta	6
5.3	Przykładowe przypadki testowe	6
6	Informacja uruchomieniowa i wdrożeniowa	7
6.1	Wymagania	7
6.2	Uruchomienie lokalne backendu	7
6.3	Uruchomienie bazy PostgreSQL w Dockerze	7
6.4	Uruchomienie całego backendu przez docker-compose	8
6.5	Uruchomienie aplikacji Flutter	8
6.6	Wdrożenie na maszynie wirtualnej ZTI	9
7	Podstawowy podręcznik użytkownika	9
7.1	Rejestracja konta	9
7.2	Logowanie	9
7.3	Utworzenie grupy	9
7.4	Dołączenie do grupy kodem	9
7.5	Dodanie produktu do listy zakupów	9
7.6	Oznaczenie produktu jako kupione	9
7.7	Dodanie posiłku do planu	10
7.8	Dodanie przepisu	10
8	Podsumowanie	10

1 Cel i zakres aplikacji

Celem projektu jest przygotowanie aplikacji wspierającej wspólne planowanie posiłków oraz organizację zakupów spożywczych. System jest przeznaczony dla osób działających w jednej grupie, na przykład domowników, współlokatorów lub małego zespołu, który chce utrzymywać wspólny plan tygodnia i listę produktów do kupienia.

Zakres projektu obejmuje część serwerową, aplikację kliencką oraz środowisko uruchomieniowe. Backend udostępnia REST API, obsługuje logikę biznesową, autoryzację i zapis danych w PostgreSQL. Frontend jest aplikacją Flutter komunikującą się z API. Wdrożenie backendu i bazy danych zostało przygotowane z użyciem kontenerów Docker oraz pliku `docker-compose.yml`.

Najważniejsze cele funkcjonalne aplikacji:

- obsługa kont użytkowników i profilu,
- tworzenie grup oraz dołączanie do grup kodem zaproszenia,
- planowanie posiłków w układzie tygodniowym,
- prowadzenie wspólnej listy zakupów,
- dodawanie i przeglądanie przepisów,
- rejestrowanie wybranych zdarzeń systemowych po stronie backendu.

2 Opis funkcjonalności aplikacji

2.1 Zarządzanie użytkownikami

Aplikacja umożliwia rejestrację nowego użytkownika, logowanie, wylogowanie oraz edycję podstawowych danych profilu. Po poprawnym zalogowaniu użytkownik otrzymuje token JWT, który jest używany przy kolejnych żądaniach wymagających autoryzacji.

Wylogowanie ma charakter bezstanowy. Oznacza to, że klient usuwa zapisany token, a backend udostępnia endpoint zamykający przepływ logowania po stronie aplikacji.

[Miejsce na zrzut ekranu: ekran logowania]

2.2 Grupy użytkowników

Podstawową jednostką współdzielenia danych jest grupa. Użytkownik może utworzyć własną grupę, a następnie przekazać innym osobom kod zaproszenia. Osoby posiadające kod mogą dołączyć do grupy i współdzielić jej listę zakupów, przepisy oraz plan posiłków.

System obsługuje role członków grupy, w tym właściciela grupy. Operacje na danych grupowych są wykonywane wyłącznie w kontekście grupy, do której użytkownik należy.

2.3 Planowanie posiłków

Moduł planowania posiłków pozwala prowadzić tygodniowy plan. Użytkownik może dodać posiłek na konkretny dzień, określić typ posiłku oraz opcjonalnie powiązać go z przepisem. Dostępne są operacje dodawania, edycji i usuwania zaplanowanych posiłków.

Plan tygodniowy pozwala szybko sprawdzić, jakie posiłki są zaplanowane w najbliższych dniach i ograniczyć powtarzające się ustalenia między członkami grupy.

[Miejsce na zrzut ekranu: tygodniowy plan posiłków]

2.4 Lista zakupów

Lista zakupów umożliwia dodawanie produktów do wspólnej listy grupy. Każdy produkt może zostać oznaczony jako kupiony, co ułatwia bieżącą współpracę między użytkownikami. Produkty można również usuwać z listy.

Funkcjonalność jest przydatna zwłaszcza wtedy, gdy kilka osób korzysta z jednej listy i aktualizuje jej stan w różnym czasie.

2.5 Przepisy

Moduł przepisów pozwala dodawać i przeglądać przepisy kulinarne. Przepis może zawierać nazwę, opis oraz składniki. Wybrany przepis może zostać użyty podczas dodawania posiłku do planu, dzięki czemu planowanie posiłków jest powiązane z bazą przepisów.

2.6 Rejestrowanie zdarzeń systemowych przez Spring AOP

Backend wykorzystuje mechanizm Spring AOP do rejestrowania wybranych zdarzeń systemowych. Operacje oznaczone adnotacją audytową są przechwytywane przez aspekt, a następnie zapisywane jako log audytowy. Pozwala to oddzielić logikę rejestrowania zdarzeń od właściwej logiki biznesowej serwisów.

3 Część serwerowa

3.1 Spring Boot REST API

Część serwerowa została zaimplementowana w Java Spring Boot. API jest podzielone na kontrolery odpowiadające modułom aplikacji:

Kontroler	Zakres odpowiedzialności
AuthController	Rejestracja, logowanie oraz wylogowanie użytkownika.
UserController	Pobieranie i aktualizacja danych profilu użytkownika.
GroupController	Tworzenie grup, dołączanie kodem zaproszenia, lista grup i członków.
MealPlanController	Obsługa planu posiłków oraz zaplanowanych posiłków.

ShoppingItemController	Dodawanie, aktualizacja i usuwanie elementów listy zakupów.
RecipeController	Dodawanie i pobieranie przepisów oraz ich składników.

API działa domyślnie na porcie 8080. Dokumentacja OpenAPI jest udostępniana pod adresem `/swagger-ui.html`, a definicja API pod `/v3/api-docs`.

3.2 Spring Data JPA i Hibernate

Warstwa dostępu do danych została przygotowana z użyciem Spring Data JPA oraz Hibernate. Encje reprezentują między innymi użytkownika, grupę, członkostwo w grupie, plan posiłków, zaplanowany posiłek, przepis, składnik przepisu, element listy zakupów oraz wpis audytowy.

Repozytoria JPA odpowiadają za komunikację z bazą danych, natomiast logika biznesowa znajduje się w serwisach, takich jak `AuthService`, `GroupService`, `MealPlanService`, `RecipeService` i `ShoppingItemService`.

3.3 PostgreSQL

Dane aplikacji są zapisywane w bazie PostgreSQL. W konfiguracji lokalnej domyślna baza nosi nazwę `wsp_db`, użytkownik to `postgres`, a hasło to `postgres`. W środowisku kontenerowym backend łączy się z bazą przez nazwę usługi `db`.

Najważniejsze ustawienia aplikacji:

```
server.port=8080
spring.datasource.url=jdbc:postgresql://localhost:5432/wsp_db
spring.jpa.hibernate.ddl-auto=update
springdoc.swagger-ui.path=/swagger-ui.html
```

3.4 JWT i Spring Security

Uwierzytelnianie opiera się na tokenach JWT. Po rejestracji lub logowaniu backend zwraca token, który klient przesyła w nagłówku:

```
Authorization: Bearer <token>
```

Spring Security filtruje żądania i sprawdza poprawność tokena. Endpointy publiczne obejmują przede wszystkim rejestrację i logowanie, natomiast operacje na grupach, planach, przepisach, profilu i liście zakupów wymagają uwierzytelnienia.

3.5 Spring AOP do logowania zdarzeń

Rejestrowanie zdarzeń zostało zrealizowane przez aspekt `AuditAspect` oraz adnotację `Auditable`. Dzięki temu metody biznesowe mogą być oznaczane jako audytowane, a sam zapis zdarzenia odbywa się poza główną logiką serwisu. Wpisy audytowe są przechowywane w encji `AuditLog`.

4 Część kliencka

4.1 Flutter

Część kliencka została przygotowana jako aplikacja Flutter. Struktura kodu jest podzielona według funkcji aplikacji: autoryzacja, grupy, plan posiłków, przepisy, lista zakupów, profil oraz ekran główny. Taki podział ułatwia rozwijanie kolejnych modułów.

4.2 Komunikacja z REST API

Komunikacja z backendem odbywa się przez klasę `ApiClient`, która obsługuje metody HTTP GET, POST, PUT, PATCH i DELETE. Klient koduje dane jako JSON i obsługuje odpowiedzi błędów zwracane przez API.

Domyślny adres backendu skonfigurowany w aplikacji klienckiej wskazuje na maszynę wirtualną:

```
http://172.20.41.12:8080
```

4.3 Logowanie i przechowywanie tokena JWT

Po zalogowaniu token JWT jest zapisywany po stronie klienta przy użyciu pakietu `flutter_secure_storage`. Przy żądaniach wymagających autoryzacji token jest odczytywany i dodawany do nagłówka `Authorization`.

4.4 Podstawowe ekrany aplikacji

Najważniejsze ekrany aplikacji:

- ekran logowania i rejestracji,
- ekran główny z nawigacją między modułami,
- ekran grup i kodu zaproszenia,
- ekran tygodniowego planu posiłków,
- ekran listy zakupów,
- ekran przepisów,
- ekran profilu użytkownika.

[Miejsce na zrzut ekranu: ekran listy zakupów]

[Miejsce na zrzut ekranu: ekran przepisów]

5 Prezentacja wybranych testów

5.1 Testy backendu

Backend zawiera testy jednostkowe i integracyjne dla najważniejszych serwisów. W projekcie znajdują się między innymi testy:

- `GroupServiceTest`,
- `MealPlanServiceTest`,
- `RecipeServiceTest`,
- `ShoppingItemServiceTest`,
- `AuditAspectIntegrationTest`,
- `WspApplicationTests`.

Testy backendu sprawdzają reguły biznesowe, poprawność operacji na danych oraz działanie mechanizmu audytu. W środowisku testowym wykorzystywana jest baza H2, co pozwala uruchamiać testy bez zewnętrznej instancji PostgreSQL.

5.2 Testy klienta

Po stronie Fluttera dostępny jest przykładowy test widgetów w katalogu `test`. Oprócz testów automatycznych wykonano testy ręczne najważniejszych scenariuszy użytkownika, ponieważ część funkcji wymaga współpracy z uruchomionym backendem i bazą danych.

5.3 Przykładowe przypadki testowe

Nazwa testu	Opis	Oczekiwany rezultat	Status
Rejestracja użytkownika	Utworzenie konta z poprawnymi danymi.	Konto zostaje zapisane, a API zwraca token JWT.	OK
Logowanie użytkownika	Logowanie poprawnym adresem e-mail i hasłem.	Użytkownik przechodzi do aplikacji, token zostaje zapisany.	OK
Utworzenie grupy	Zalogowany użytkownik tworzy nową grupę.	Grupa pojawia się na liście, generowany jest kod zaproszenia.	OK
Dołączenie kodem	Drugi użytkownik wpisuje kod zaproszenia.	Użytkownik zostaje członkiem tej samej grupy.	OK
Dodanie produktu	Dodanie produktu do listy zakupów grupy.	Produkt jest widoczny na wspólnej liście.	OK
Oznaczenie produktu	Zmiana statusu produktu na kupiony.	Element listy zmienia stan i pozostaje widoczny z nowym statusem.	OK

Dodanie po- siłku	Dodanie posiłku do wybra- nego dnia tygodnia.	Posiłek pojawia się w pla- nie tygodniowym.	OK
Dodanie prze- pisu	Utworzenie przepisu ze skład- nikami.	Przepis jest widoczny na liście i może zostać użyty w planie.	OK
Audyt operacji	Wykonanie operacji oznaczo- nej jako audytowana.	W bazie powstaje wpis audytowy.	OK

6 Informacja uruchomieniowa i wdrożeniowa

6.1 Wymagania

Do uruchomienia projektu wymagane są:

- Java 21,
- Maven lub wrapper Maven dołączony do projektu,
- Docker,
- Docker Compose,
- Flutter SDK,
- dostęp do portów 8080 i 5432.

6.2 Uruchomienie lokalne backendu

Backend można uruchomić lokalnie z katalogu `backend/wsp`. Przed startem aplikacji musi działać baza PostgreSQL zgodna z konfiguracją w pliku `application.properties`.

```
cd backend/wsp
./mvnw spring-boot:run
```

Uruchomienie testów backendu:

```
cd backend/wsp
./mvnw test
```

6.3 Uruchomienie bazy PostgreSQL w Dockerze

Samą bazę danych można uruchomić z wykorzystaniem usługi db z pliku `docker-compose.yml`:

```
cd backend/wsp
docker-compose up -d db
```

Po uruchomieniu baza jest dostępna lokalnie na porcie 5432.

6.4 Uruchomienie całego backendu przez docker-compose

Najprostszy sposób uruchomienia backendu razem z bazą danych to użycie Docker Compose:

```
cd backend/wsp
docker-compose up -d --build
docker ps
docker logs -f ZTI-wsp-backend
```

Fragment konfiguracji `docker-compose.yml`:

```
services:
  db:
    image: postgres:16
    container_name: ZTI-wsp-postgres
    environment:
      POSTGRES_DB: wsp_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    ports:
      - "5432:5432"

  backend:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: ZTI-wsp-backend
    depends_on:
      - db
    environment:
      SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/wsp_db
      SPRING_DATASOURCE_USERNAME: postgres
      SPRING_DATASOURCE_PASSWORD: postgres
    ports:
      - "8080:8080"
```

6.5 Uruchomienie aplikacji Flutter

Frontend można uruchomić z katalogu `client/wsp`. Aplikacja komunikuje się z backendem pod adresem skonfigurowanym w kliencie API.

```
cd client/wsp
flutter pub get
flutter run
```

Testy klienta:

```
cd client/wsp
flutter test
```

6.6 Wdrożenie na maszynie wirtualnej ZTI

Aplikacja została przygotowana do pracy na maszynie wirtualnej ZTI o adresie IP:

```
172.20.41.12
```

Na maszynie wirtualnej uruchamiane są kontenery backendu oraz bazy danych PostgreSQL. Backend jest wystawiany na porcie 8080, a aplikacja Flutter komunikuje się z adresem `http://172.20.41.12:8080`.

Przykładowa procedura wdrożenia:

```
cd backend/wsp
docker-compose up -d --build
docker ps
docker logs -f ZTI-wsp-backend
```

7 Podstawowy podręcznik użytkownika

7.1 Rejestracja konta

Użytkownik otwiera aplikację, wybiera rejestrację i podaje wymagane dane konta. Po poprawnym utworzeniu konta może przejść do korzystania z aplikacji.

7.2 Logowanie

Na ekranie logowania użytkownik wpisuje adres e-mail i hasło. Po poprawnym uwierzytelnieniu aplikacja zapisuje token sesji i przechodzi do ekranu głównego.

7.3 Utworzenie grupy

Po zalogowaniu użytkownik przechodzi do ekranu grup i wybiera opcję utworzenia grupy. Następnie podaje nazwę grupy. System tworzy grupę i wyświetla kod zaproszenia.

7.4 Dołączenie do grupy kodem

Użytkownik, który otrzymał kod zaproszenia, przechodzi do ekranu grup i wpisuje kod. Po zatwierdzeniu zostaje dodany do grupy i uzyskuje dostęp do jej danych.

7.5 Dodanie produktu do listy zakupów

Na ekranie listy zakupów użytkownik wybiera dodanie produktu, wpisuje nazwę oraz opcjonalne szczegóły. Produkt zostaje zapisany na wspólnej liście grupy.

7.6 Oznaczenie produktu jako kupione

Na liście zakupów użytkownik wybiera produkt i zmienia jego status na kupiony. Pozostali członkowie grupy widzą zaktualizowany stan listy.

7.7 Dodanie posiłku do planu

Na ekranie planu posiłków użytkownik wybiera dzień oraz typ posiłku, a następnie podaje nazwę posiłku lub wybiera przepis. Po zapisaniu pozycja pojawia się w tygodniowym planie.

7.8 Dodanie przepisu

Na ekranie przepisów użytkownik wybiera dodanie przepisu, uzupełnia nazwę, opis oraz składniki. Przepis może później zostać wykorzystany przy planowaniu posiłku.

8 Podsumowanie

Projekt realizuje kompletny przepływ pracy związany ze wspólnym planowaniem posiłków i zakupów. Backend Spring Boot odpowiada za REST API, bezpieczeństwo, logikę biznesową i zapis danych w PostgreSQL. Aplikacja Flutter zapewnia mobilny interfejs użytkownika oraz komunikację z API z użyciem tokena JWT. Docker Compose upraszcza uruchamianie backendu i bazy danych zarówno lokalnie, jak i na maszynie wirtualnej ZTI.

Najważniejsze założenia projektu zostały spełnione: użytkownicy mogą tworzyć grupy, współdzielić listę zakupów, planować posiłki, korzystać z przepisów oraz pracować na wspólnych danych przechowywanych po stronie serwera.